TECHNICAL DOCUMENTATION

Twittus_Extractorium

Python 3.8+ | Playwright | SQLite | Ethical Use Only

Version **1.0** | April 18, 2026

Abstract. *Twittus_Extractorium* is a browser-automation-based scraper for collecting publicly accessible tweets from X (formerly Twitter). Built on **Playwright** and **SQLite**, it supports profile scraping, keyword search scraping, deduplication, and human-like rate limiting. It is intended exclusively for academic research, public-data analysis, and journalistic investigation of publicly posted content.

Language	Python 3.8 or later
Browser engine	Chromium via Playwright
Storage	SQLite (file-based, zero server required)
License	For research/academic use only

Contents

§1 Introduction

Twittus_Extractorium was designed to fill a pragmatic gap in social-media research tooling: the official X API is rate-limited, expensive, and restricted, yet a large body of publicly accessible content exists that researchers legitimately need to study.

The scraper automates a real Chromium browser session through Playwright, closely mimicking human browsing behaviour — including randomised delays, gradual scrolling, and a visible (non-headless) browser window — to gather tweet metadata without relying on undocumented API endpoints.

1.1 Goals

- Provide a single-login, multi-task batch scraping workflow.
- Store results in a local, portable SQLite database with automatic deduplication.
- Expose a clean Python API so downstream analysis scripts can invoke scraping as a library call.
- Maintain ethical use through rate-limiting, public-content-only access, and clear user disclaimers.

1.2 Non-Goals

Scope Limitations

Twittus_Extractorium does **not**:

- Access private accounts, DMs, or any content requiring special credentials.
- Bypass CAPTCHA or authentication challenges programmatically.
- Operate without a human confirming the login step.
- Support headless operation (headless mode cannot complete manual login).

§2 Architecture Overview

2.1 Component Diagram

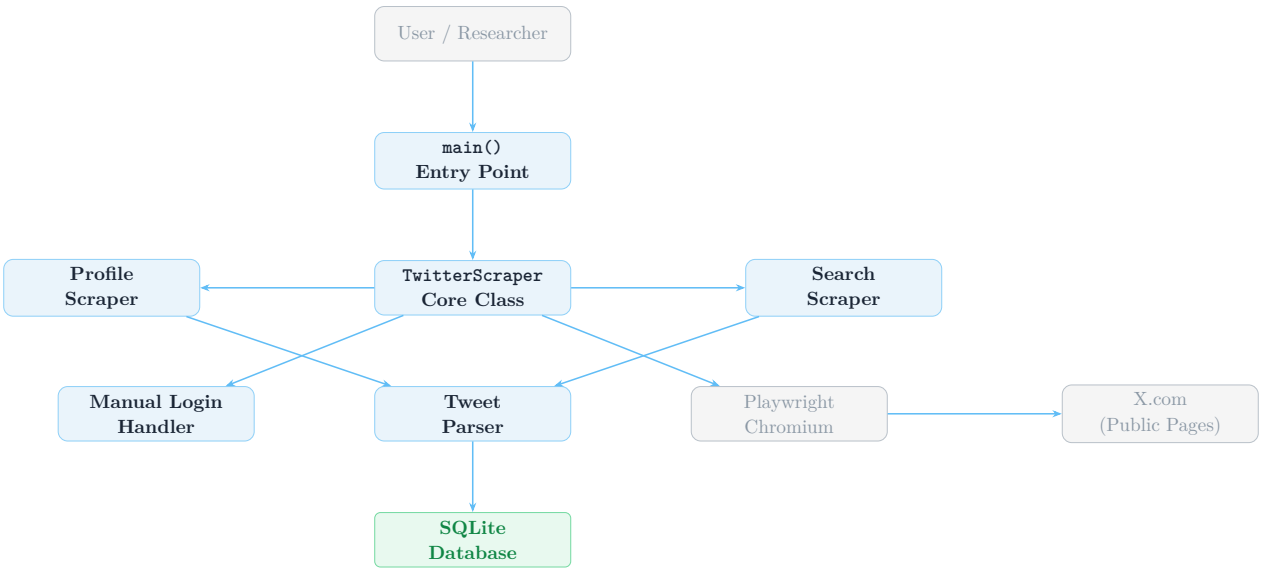

Figure 1: High-level component interaction for *Twittus_Extractorium*.

2.2 Data Flow

1. The user defines a list of **tasks** (profile scrapes and/or keyword searches).
2. A single Chromium browser instance is launched; the user logs in manually.
3. For each task, the scraper navigates to the target URL.
4. Playwright locates `article[data-testid="tweet"]` elements.
5. Each element is parsed by `parse_tweet()` into a structured dictionary.
6. New records are persisted to SQLite via `save_tweets()`; duplicates are silently skipped.
7. Human-like delays are injected between scrolls and between tasks.

§3 Installation & Requirements

3.1 Prerequisites

Dependency	Version	Purpose
Python	>= 3.8	Runtime
playwright	>= 1.40	Browser automation
Chromium browser	(via playwright install)	Page rendering
SQLite	bundled with Python	Data persistence

3.2 Step-by-Step Setup

1 — Create a virtual environment

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
```

2 — Install Python dependencies

```
pip install playwright
```

3 — Install Playwright browser binaries

```
playwright install chromium
```

4 — Run the scraper

```
python twittus_extractorium.py
```

Virtual Environment

Always activate the virtual environment before running the scraper. The script will fail silently if Playwright cannot find the Chromium binary.

§4 Module Reference

4.1 Class `TwitterScraper`

The main class that orchestrates the browser session, scraping logic, and database persistence. All public methods are described below.

4.1.1 Constructor

`__init__`

```
def __init__(self, db_path: str = "twitter_data.db",
             headless: bool = False)
```

Parameter	Type	Description
`db_path`	`str`	File path for the SQLite database. Created automatically if absent.
`headless`	`bool`	Must be `False` for manual login. Headless mode is incompatible with interactive login.

4.1.2 `wait_for_manual_login(page)`

Navigates to the X login page and pauses execution until the user presses `ENTER` in the console. Performs post-login verification by checking for multiple DOM indicators (primary column, account switcher, post button).

Returns: `bool` — `True` if login is confirmed, `False` otherwise.

4.1.3 `setup_database()`

Creates the `tweets` table and two indices (`author_username`, `scraped_at`) if they do not already exist.

4.1.4 `save_tweets(tweets)`

`save_tweets`

```
def save_tweets(self, tweets: List[Dict]) -> None
```

Inserts tweet records into the database using `INSERT OR IGNORE` to prevent duplicates keyed on `tweet_id`. Logs counts of new saves and skipped duplicates.

4.1.5 `human_delay(min_seconds, max_seconds)`

Sleeps for a random duration drawn uniformly from `[min_seconds, max_seconds]`. Default window is **1.5 – 4.0 s**.

4.1.6 `scroll_page(page, scrolls)`

Scrolls the page `scrolls` times by a randomised pixel offset (500 – 900 px per scroll), waiting 1.5 – 3.0 s between each to allow lazy-loaded content to appear.

4.1.7 extract_number(text)

Converts human-readable engagement counts (`"1.2K"`, `"500"`, `"1M"`) to integers. **Returns:** `int`

4.1.8 extract_tweet_id(url)

Parses the tweet ID from a status URL via the regex `/status/(\d+)`. **Returns:** `Optional[str]`

4.1.9 parse_tweet(article_element, source_type, source_query)

The core extraction routine. Given a Playwright element handle for a tweet `<article>`, it extracts:

- Tweet URL and ID (via `<time>` ancestor link)
- Author username (from URL path)
- Display name (first non-`@` span inside `[data-testid="User-Name"]`)
- Tweet text (`[data-testid="tweetText"]`)
- Engagement counts: likes, replies, retweets (from `aria-label` attributes)
- ISO 8601 timestamp (`<time datetime>`)

Returns: `Optional[Dict]` — returns `None` if the element cannot be parsed.

4.1.10 scrape_profile(username, max_scrolls, ...)

scrape_profile

```
def scrape_profile(
    self,
    username: str,
    max_scrolls: int = 5,
    page: Page = None,
    context = None,
    browser = None
) -> List[Dict]
```

Navigates to `https://x.com/{username}` and collects tweets across `max_scrolls` scroll cycles. Detects inaccessible profiles (suspended/non-existent) before scraping.

Parameter	Default	Notes
`username`	—	Without the `@` prefix
`max_scrolls`	`5`	Each scroll fires 2 sub-scrolls
`page`	`None`	Pass an existing page to reuse session
`context`	`None`	Pass for session reuse
`browser`	`None`	Pass for session reuse

4.1.11 scrape_search(query, max_scrolls, ...)

Analogous to `scrape_profile` but navigates to the URL `https://x.com/search` with parameters `q` and `f=live` (live/latest tab). The query is URL-encoded via `urllib.parse.quote`.

4.1.12 scrape_with_single_login(tasks)

scrape_with_single_login

```
def scrape_with_single_login(self, tasks: List[Dict]) -> None
```

The recommended entry-point for batch usage. Launches a single browser, performs one login, then iterates over all tasks, calling either scrape_profile or scrape_search depending on the task's type key.

Task schema:

Key	Type	Values
type	str	"profile" or "search"
target	str	Username (no @) or search term
max_scrolls	int	Any positive integer (default 5)

§5 Database Schema

All scraped data is stored in a single SQLite database file (default: twitter_data.db).

5.1 Table: tweets

Column	Type	Description
tweet_id	TEXT (PK)	Unique numeric ID extracted from tweet URL
author_username	TEXT	Twitter handle (without @)
author_display_name	TEXT	Display name as shown on the profile
tweet_text	TEXT	Full plaintext body of the tweet
tweet_url	TEXT	Canonical URL (https://x.com/...)
like_count	INTEGER	Parsed like count at time of scrape
reply_count	INTEGER	Parsed reply count
retweet_count	INTEGER	Parsed retweet count (includes quote-tweets)
timestamp	TEXT	ISO 8601 original post time
scraped_at	TEXT	ISO 8601 scrape timestamp
source_type	TEXT	"profile" or "search"
source_query	TEXT	Username or search query that yielded the tweet

5.2 Indices

- idx_author_username on author_username — fast per-user queries.
- idx_scraped_at on scraped_at — fast recency-based queries.

5.3 Querying the Database

Example queries

```python
import sqlite3

conn = sqlite3.connect("twitter_data.db")
cursor = conn.cursor()

# All tweets from a specific user
cursor.execute(
    "SELECT tweet_text, like_count FROM tweets "
    "WHERE author_username = ? ORDER BY like_count DESC",
    ("elonmusk",)
)

# Top 10 most-liked tweets in the dataset
cursor.execute(
    "SELECT author_username, tweet_text, like_count FROM tweets "
    "ORDER BY like_count DESC LIMIT 10"
)

# Summary statistics
cursor.execute(
    "SELECT source_query, COUNT(*) AS n, AVG(like_count) AS avg_likes "
    "FROM tweets GROUP BY source_query"
)

conn.close()
```

§6 Usage Guide

6.1 Quick Start

Minimal usage example

```python
from twittus_extractorium import TwitterScraper

scraper = TwitterScraper(db_path="my_research.db", headless=False)

tasks = [
    {"type": "profile", "target": "NASA",          "max_scrolls": 8},
    {"type": "search",  "target": "climate Nepal", "max_scrolls": 20},
    {"type": "search",  "target": "#OpenSource",   "max_scrolls": 10},
]

scraper.scrape_with_single_login(tasks)
```

Login Flow

When the script runs, a Chromium window will open and navigate to `x.com/i/flow/login`. Complete the login in the browser, then press `ENTER` in the terminal. The scraper will verify the session and proceed automatically.

6.2 Single-Task Mode

You can invoke profile or search scraping independently if you manage the browser session yourself:

Single profile scrape

```python
scraper = TwitterScraper()

# Returns a list of tweet dicts -- not auto-saved
tweets = scraper.scrape_profile("SomeUser", max_scrolls=3)

# Manually save to DB
scraper.save_tweets(tweets)
```

6.3 Choosing `max_scrolls`

Goal	Recommended `max_scrolls`	Approx. tweets
Quick sample	3 – 5	30 – 80
Standard research session	10 – 15	100 – 250
Deep longitudinal collection	30 – 50	400 – 800
Comprehensive keyword harvest	50 – 100	800 – 2000

Tweet counts vary widely depending on reply volume, retweet density, and X's own feed algorithm at the time of scraping.

§7 Configuration Reference

7.1 Constructor Parameters

Parameter	Default	Notes
`db_path`	`"twitter_data.db"`	Relative or absolute path. File is created on first run.
`headless`	`False`	**Must remain `False`** — headless mode cannot handle interactive login.

7.2 Tunable Constants (in source)

Location	Default	Effect
`human_delay` min/max	1.5 / 4.0 s	Inter-action pause
`scroll_page` offset	500 – 900 px	Pixels per sub-scroll
`scrape_with_single_login` inter-task delay	5 – 10 s	Pause between tasks

§8 Ethical Guidelines & Legal Considerations

Read Before Use

Failure to comply with these guidelines may violate X's Terms of Service and applicable laws. The authors accept no liability for misuse.

8.1 Permitted Uses

- Academic research into public discourse, misinformation, or social trends.
- Journalistic investigation of public figures' public statements.
- Non-commercial content analysis with proper ethical-review board approval.

8.2 Prohibited Uses

- Targeted harassment or doxxing of individuals.
- Building spam lists or unsolicited marketing databases.
- Scraping private or protected accounts.
- Circumventing any technical protection measures.
- Reselling or redistributing scraped data commercially.

8.3 Rate Limiting & Courtesy

The scraper includes several built-in courtesy measures:

- **Randomised delays** between every action to avoid request bursts.
- **Visible browser** mode that prevents undetected automated crawling.
- **Human confirmation** at login — no credentials are stored or transmitted programmatically.
- **Inter-task pauses** of 5 – 10 seconds between scraping runs.

Best Practice

Limit any single session to a reasonable number of tasks. Excessive scraping in a single session may result in temporary rate-limiting by X.

§9 Logging

The scraper uses Python's standard `logging` module at level `INFO`. Debug-level messages (individual tweet IDs, DOM parse errors) are available by changing the log level:

Enable debug logging

```
import logging
logging.getLogger().setLevel(logging.DEBUG)
```

Log format:

```
2024-01-15 10:32:05,123 - INFO  - Scraping profile: @NASA
2024-01-15 10:32:08,456 - INFO  - Scroll 1/8
2024-01-15 10:32:08,789 - INFO  - Found 12 tweet elements on page
2024-01-15 10:32:11,234 - INFO  - Saved 11 new tweets, skipped 1 duplicates
```

§10 Troubleshooting

Symptom	Resolution
`playwright._impl._errors.Error: Executable doesn't exist`	Run `playwright install chromium` to download the browser binary.
`ModuleNotFoundError: No module named 'playwright'`	Activate the virtual environment, then `pip install playwright`.
No tweets found on a profile page	The profile may be suspended, private, or using a geo-restricted layout. Check the URL manually.
Login verification fails but user is logged in	Type `y` at the confirmation prompt; the scraper will proceed.
Engagement counts all zero	X may have changed `aria-label` attribute format. Update the regex in `extract_number`.
Duplicate tweet IDs in output list	Expected — `save_tweets` silently drops them via `INSERT OR IGNORE`.
Very slow scraping	Increase delay parameters or reduce `max_scrolls`. X may be throttling.

§11 Project Structure

Repository layout

```
twittus_extractorium/
|
|-- twittus_extractorium.py    # Main module (all code)
|-- twitter_data.db            # SQLite output (created on first run)
|-- README.md                  # Quick-start guide
|-- requirements.txt           # pip dependencies
`-- .venv/                     # Virtual environment (not committed)
```

requirements.txt

```
playwright>=1.40.0
```

§12 Extending the Scraper

12.1 Adding a New Source Type

1. Implement a new method `scrape_{source}()` that accepts `page`, `context`, and `browser` keyword arguments for session reuse.
2. Add a new branch in `scrape_with_single_login()` for your new `type` key.
3. The existing `parse_tweet()` and `save_tweets()` pipeline requires no changes.

12.2 Exporting to CSV / JSON

Export to CSV with pandas

```python
import pandas as pd
import sqlite3

conn = sqlite3.connect("twitter_data.db")
df = pd.read_sql_query("SELECT * FROM tweets", conn)
conn.close()

df.to_csv("tweets_export.csv", index=False)
df.to_json("tweets_export.json", orient="records", indent=2)
```

12.3 Integrating with Analysis Pipelines

Because `scrape_profile` and `scrape_search` return plain Python `List[Dict]` objects, they can be piped directly into any analysis framework without touching the database:

Inline analysis without database

```python
import pandas as pd
from twittus_extractorium import TwitterScraper

scraper = TwitterScraper()
tweets  = scraper.scrape_search("open source AI", max_scrolls=10)

df = pd.DataFrame(tweets)
top = df.nlargest(20, "like_count")[["author_username", "tweet_text",
                                     "like_count"]]
print(top)
```

§13 Changelog

Version	Date	Changes
1.0.0	April 18, 2026	Initial public release. Profile scraping, search scraping, SQLite persistence, single-login batch mode, human-delay rate limiting.

Disclaimer

Legal Disclaimer

Twittus_Extractorium is provided **as-is** for academic and research purposes only. Users are solely responsible for ensuring their use complies with X's Terms of Service, applicable data-protection regulations (GDPR, CCPA, etc.), and any institutional ethics requirements. The developers make no warranties, express or implied, and accept no liability for damages arising from use or misuse of this software.